

Technical Write-up Task 2

Prediksi Navigasi Wikipedia (Wikispeedia)

1. Konteks masalah

Task ini berangkat dari Wikispeedia, permainan di mana seseorang harus berpindah dari satu artikel Wikipedia ke artikel target hanya dengan mengklik hyperlink di dalam halaman. Diberikan posisi saat ini dan artikel target, kami diminta memprediksi artikel mana yang akan diklik berikutnya. Data yang tersedia adalah tabel judul untuk 4.604 artikel, tabel kategori, 9.000 baris latih yang memuat current, target, dan next sebagai label, 6.000 baris uji tanpa label, serta satu folder screenshot berisi tangkapan layar penuh tiap halaman artikel. Metrik penilaiannya adalah akurasi prediksi.

Hal yang membuat task ini menarik adalah struktur hyperlink antar artikel tidak diberikan dalam bentuk tabel. Satu-satunya tempat informasi itu ada adalah di dalam gambar screenshot. Karena itu bagian terbesar dari pekerjaan kami bukan pemodelan, melainkan merekonstruksi graf hyperlink dari piksel.

2. Ringkasan pendekatan

Solusi akhir kami tidak melatih model sama sekali. Alurnya terdiri dari tiga tahap. Pertama, seluruh hyperlink dibaca dari screenshot memakai OCR, lalu dipetakan kembali ke `article_id` untuk membentuk graf berarah. Kedua, dari graf tersebut dihitung dua skor struktural, yaitu in-degree dan PageRank, yang dikalikan menjadi satu skor hub. Ketiga, untuk tiap baris uji, kandidat jawaban adalah semua link yang benar-benar ada di halaman current, dan kandidat dipilih berdasarkan jarak terpendek ke target di graf, dengan skor hub sebagai pemecah kalau ada beberapa kandidat berjarak sama.

Dasar pemikirannya sederhana. Navigasi Wikispeedia pada dasarnya adalah usaha mencari jalur terpendek menuju artikel target, dan orang biasanya melewati artikel hub yang koneksinya banyak lebih dulu sebelum menyempit ke target. Aturan berbasis graf merepresentasikan strategi itu secara langsung, tanpa perlu mempelajarinya dari contoh yang terbatas.

3. Temuan eksplorasi yang menentukan desain

Eksplorasi tabel dilakukan lebih dulu sebelum menyentuh screenshot, dan tiga temuan berikut yang paling memengaruhi desain solusi.

- Tidak ada jawaban yang bisa dihafal dari data latih. Setiap pasangan current dan target di train unik, tidak ada satu pun pasangan uji yang pernah muncul di train, dan 240 target di data uji sama sekali tidak beririsan dengan 360 target di data latih. Aturan yang dipakai harus benar-benar general terhadap target baru, sehingga pendekatan yang mengandalkan hafalan pola target akan gagal.
- Distribusi artikel next sangat timpang dan mengarah ke hub. Dari 9.000 baris, artikel next didominasi halaman besar seperti United States yang muncul 1.048 kali atau 11,64 persen dari seluruh baris, disusul Europe dan United Kingdom. Ini konsisten dengan cara orang bernavigasi dan menjadi alasan skor hub dipakai sebagai penentu saat ada beberapa kandidat berjarak sama.
- Kategori terlalu tipis untuk diandalkan. Rata-rata satu artikel hanya punya 1,13 kategori. Artikel next memang lebih sering se-subjek dengan target dibanding current, yaitu 32,3 persen berbanding 17,2 persen, jadi sinyalnya ada, tetapi jauh dari cukup untuk jadi sinyal utama. Ini memperkuat keputusan untuk fokus pada struktur hyperlink.

Satu temuan tambahan yang berguna secara teknis, banyak artikel current dipakai ulang antara train dan test, yaitu 2.866 artikel. Artinya hasil OCR satu halaman bisa dipakai untuk banyak baris sekaligus, sehingga OCR cukup dijalankan sekali untuk seluruh 4.604 artikel.

4. Rekonstruksi graf hyperlink lewat OCR

Hyperlink di halaman Wikipedia selalu berwarna biru, jadi langkah pertama adalah memisahkan piksel biru dari sisa halaman lewat ambang batas pada kanal RGB. Piksel biru yang berdekatan

digabung menjadi satu span memakai dilasi horizontal dan connected component labeling. Dilasi horizontal sebesar 20 piksel dipilih supaya kata-kata dalam satu link yang sama menyatu, tetapi dua link berbeda yang terpisah tidak ikut menyatu. Span kemudian disaring berdasarkan tinggi dan lebar minimum untuk membuang noise.

Tiap span dipotong ketat lalu ditumpuk menjadi satu gambar komposit, sehingga OCR hanya membaca teks link dan bukan seluruh isi artikel. Percobaan awal memanggil OCR sekali per halaman ternyata sangat lambat, karena biaya tiap panggilan OCR didominasi overhead tetap dan bukan ukuran gambar. Solusinya, sekitar 15 sampai 20 halaman digabung menjadi satu komposit besar per panggilan dengan target tinggi 12.000 piksel, sehingga jumlah panggilan turun drastis. Hasil OCR dipetakan kembali ke halaman dan span asalnya berdasarkan posisi vertikal kotak deteksi.

Teks hasil OCR dipetakan ke `article_id` lewat pencocokan judul yang sudah dinormalisasi menjadi huruf kecil tanpa tanda baca. Pencocokan persis dicoba lebih dulu, baru fuzzy matching dengan ambang skor 86 kalau tidak ketemu. Ambang ini cukup longgar untuk menoleransi kesalahan baca kecil, tetapi cukup ketat untuk tidak menghasilkan pasangan yang salah. Hasil akhirnya disimpan ke cache di disk supaya tahap ini tidak perlu diulang. Graf yang terbentuk punya 4.604 node dan 86.579 edge, atau sekitar 19 link keluar per halaman, angka yang masuk akal untuk Wikipedia versi sekolah yang dipakai di kompetisi ini.

5. Aturan prediksi

Untuk satu target, jarak dari semua node lain menuju target dihitung sekali lewat breadth first search pada graf yang arahnya dibalik. Jalur dari target ke suatu artikel di graf terbalik sama persis dengan jalur dari artikel itu ke target di graf asli, sehingga satu kali BFS sudah cukup untuk mendapat jarak seluruh node sekaligus. Hasilnya disimpan di cache per target. Karena tiap target selalu muncul di 25 baris berbeda, cache ini menghemat banyak komputasi berulang.

Kandidat jawaban untuk satu baris adalah semua link hasil OCR dari halaman current. Tiap kandidat diberi skor berupa pasangan nilai, yaitu jarak ke target dan negatif dari skor hub. Kandidat dengan skor terkecil dipilih, artinya jarak terdekat ke target diutamakan lebih dulu, dan skor hub baru dipakai ketika beberapa kandidat punya jarak yang sama. Kandidat yang tidak bisa mencapai target diberi jarak sentinel 999 sehingga selalu kalah dari kandidat yang terhubung. Kalau OCR tidak menemukan link sama sekali di suatu halaman, yang jarang terjadi, seluruh artikel dipakai sebagai kandidat cadangan.

Perlu ditegaskan bahwa skor hub dihitung murni dari struktur graf dan tidak memakai kolom `next_article_id` sama sekali. Aturan prediksi juga tidak melihat label. Ini yang membuat pendekatannya aman terhadap kenyataan bahwa target di data uji sepenuhnya berbeda dari target di data latih.

6. Validasi

Karena tidak ada model yang dilatih, fungsi prediksi bersifat deterministik terhadap graf, current, dan target, sehingga tidak ada risiko overfitting maupun kebocoran label dari train ke test. Konsekuensinya, akurasi pada seluruh 9.000 baris data latih sudah menjadi estimasi yang wajar untuk performa di data uji, tanpa perlu skema validasi silang. Sebagai pembanding kami memakai baseline naif yang selalu menebak satu artikel hub paling populer.

7. Hasil

Pendekatan	Akurasi
Baseline hub tunggal, selalu menebak artikel terpopuler	11,64 persen
Aturan graf, jarak BFS ditambah skor hub (train, 9.000 baris)	60,40 persen
Submission di leaderboard	0,620

Akurasi 60,40 persen di data latih jauh melampaui baseline hub tunggal yang hanya 11,64 persen. Angka baseline itu sendiri persis sama dengan porsi baris yang next-nya memang artikel paling populer, sesuai temuan di tahap eksplorasi. Selisih yang besar menunjukkan jarak ke target di graf memberi sinyal yang jauh lebih kuat dibanding sekadar menebak artikel populer.

Skor leaderboard sebesar 0,620 konsisten dengan akurasi data latih 60,40 persen, bahkan sedikit lebih tinggi. Kesesuaian ini memang diharapkan, karena aturan prediksi tidak pernah menyentuh label, sehingga tidak ada mekanisme yang membuat performanya di data latih menjadi optimistis. Ini sekaligus menjadi konfirmasi bahwa pendekatannya benar-benar general terhadap target yang belum pernah muncul.

8. Keterbatasan dan arah pengembangan

Kualitas prediksi dibatasi langsung oleh kualitas OCR. Setiap link yang gagal terbaca berarti kandidat yang hilang dan jarak graf yang terlalu besar. Perbaikan paling menjanjikan ada di tahap ini, misalnya lewat penyempurnaan segmentasi span untuk link yang terpotong antar baris. Arah lain yang wajar adalah memanfaatkan posisi vertikal link di halaman, yang sudah ikut kami simpan saat OCR, karena link yang berada di bagian atas artikel kemungkinan lebih sering diklik. Selain itu, saat beberapa kandidat berjarak sama, sinyal kesamaan kategori dengan target bisa dipakai sebagai pemecah tambahan di samping skor hub.

9. Waktu eksekusi dan dependensi

Tahap OCR adalah bagian paling berat karena harus membaca ribuan screenshot dan dijalankan di GPU. Setelah hasil OCR tersedia di cache, seluruh tahap sisanya, yaitu membangun graf, menghitung PageRank, BFS, prediksi, dan penulisan submission, selesai dalam hitungan detik. Prediksi untuk 9.000 baris latih memakan waktu sekitar 1,7 detik. Dependensi di luar bawaan lingkungan Kaggle hanya easyocr, rapidfuzz, dan networkx. Notebook menjalankan seluruh alur ini secara berurutan dari nol.